

Лабораторная работа №9 – «Многоязыковое программирование»

Цель: программное обеспечение, разработанное по технологиям многоязыкового программирования и эффективно действующее современную целевую архитектуру.

Задачи:

- 1) Разработать техническое задание на программное обеспечение
- 2) Разработать динамически подключаемый код по обработке данных
- 3) Разработать головную программу
- 4) Организовать эффективную обработку данных
- 5) Разработать многопоточный векторизованный собственный код с использованием выравнивания адресов

Техническое задание

Инструментальная платформа: GNU/Linux (Ubuntu 24.04)

Целевая платформа: GNU/Linux

Динамическая библиотека: C++ (собственный код)

Головная программа: Python (отличный от C++ язык) с графическим интерфейсом (Tkinter)

Обработка: Умножение матриц размером 1024x1024

Создание структуры проекта

```
mkdir LAB9
cd LAB9
mkdir src lib include tests
```

- Makefile – файл сборки;
- main.py - головная программа на Python;
- include/
 - matrix_ops.h - заголовочный файл с функциями библиотеки;
- lib/
 - libmatrix.so – динамическая библиотека;
- src/
 - matrix_ops.cpp - реализация функций библиотеки;
- tests/
 - test_matrix – исполняемый файл;
 - test_matrix.cpp - тестовая программа;

Заголовочный файл `include/matrix_ops.h`

```
#ifndef MATRIX_OPS_H
#define MATRIX_OPS_H

#ifdef __cplusplus
extern "C" {
#endif

// Структура для хранения матрицы
typedef struct
{
    float* data;
    int rows;
    int cols;
} Matrix;

// Функции для работы с матрицами
Matrix* create_matrix(int rows, int cols);
void free_matrix(Matrix* m);

// Различные версии умножения матриц
void matrix_multiply_naive(const Matrix* A, const Matrix* B, Matrix* C);
void matrix_multiply_optimized(const Matrix* A, const Matrix* B, Matrix* C);
void matrix_multiply_vectorized(const Matrix* A, const Matrix* B, Matrix* C);
void matrix_multiply_aligned(const Matrix* A, const Matrix* B, Matrix* C);
void matrix_multiply_parallel(const Matrix* A, const Matrix* B, Matrix* C);

// Вспомогательные функции
void init_random(Matrix* m);
double get_time_ms();

#ifdef __cplusplus
}
#endif

#endif
```

- C++ использует name mangling – переименовывает функции, добавляя информацию о типах аргументов. Python программа через ctypes ожидает найти в библиотеке символы в C-стиле (без манглинга). Без этого Python не сможет найти функции!

C: `matrix_multiply_naive` -> `matrix_multiply_naive`

C++: `matrix_multiply_naive` -> `_Z21matrix_multiply_naivePK6MatrixS0_P6`

- `__cplusplus` – макрос, который автоматически определяется только компилятором C++; для компилятора C блок игнорируется, а для C++ включается C-совместимый режим именования;
- `extern "C"` – использование C-стиля именования функций;

Файл реализации src/matrix_ops.cpp

1. Создание матрицы с выравниванием

```
Matrix* create_matrix(int rows, int cols)
{
    Matrix* m = (Matrix*)malloc(sizeof(Matrix));
    if (!m)
    {
        return nullptr;
    }

    m->rows = rows;
    m->cols = cols;

    int ret = posix_memalign((void**)&m->data, 64, rows * cols * sizeof(float));
    if (ret != 0)
    {
        free(m);
        return nullptr;
    }

    memset(m->data, 0, rows * cols * sizeof(float));

    return m;
}
```

- `int posix_memalign(void **memptr, size_t alignment, size_t size);`
 - `void** memptr` – указатель на указатель (куда записать адрес выделенной памяти);
 - `size_t alignment` – требуемое выравнивание (степень двойки);
 - `size_t size` – размер выделяемой памяти в байтах;
- Обычный `malloc` выделяет память с выравниванием, подходящим для любого типа данных (обычно 8 или 16 байт), но не гарантирует выравнивание для SIMD-инструкций.
- **SIMD** (Single Instruction, Multiple Data) - это технология, позволяющая процессору выполнять одну операцию над несколькими элементами данных одновременно. Если обычные (скалярные) инструкции обрабатывают по одному числу за раз, то SIMD-инструкции работают с целыми блоками (векторами) данных, что значительно ускоряет вычисления. Вместо того чтобы в цикле складывать пары чисел по очереди, процессор загружает их в специальные широкие регистры (128, 256 или 512 бит) и выполняет сложение всех пар за один такт.
- **AVX** (Advanced Vector Extensions) - это расширение SIMD-инструкций, которое значительно увеличивает «ширину» данных, обрабатываемых процессором за один раз.
- **SSE** (Streaming SIMD Extensions) – это SIMD-набор инструкций, который появился в процессорах Intel Pentium III в 1999 году.

Выравнивание	SIMD	Размер регистра	Количество float
16 байт	SSE	128 бит	4
32 байта	AVX, AVX2	256 бит	8
64 байта	AVX-512	512 бит	16

- AVX-инструкции ожидают, что адрес будет кратен 32. Если это не так, то процессор может работать медленнее (невыровненный доступ) и некоторые инструкции могут вызывать исключение.
- `void* memset(void* ptr, int value, size_t num);`
 - `void* ptr` – указатель на начало памяти;
 - `int value` – значение для записи;
 - `size_t num` – количество байтов для заполнения;
- Почему не цикл? Функция `memset` оптимизирована (написана на ассемблере), она использует SIMD-инструкции для быстрого заполнения.

2. Освобождение памяти, занимаемой матрицей

```
void free_matrix(Matrix* m)
{
    if (m)
    {
        free(m->data);
        free(m);
    }
}
```

- `void free(void* ptr)` – функция из стандартной библиотеки C (`stdlib.h`), которая возвращает ранее выделенную память обратно в кучу (heap).

3. Инициализация матрицы случайными числами

```
void init_random(Matrix* m)
{
    srand(time(nullptr));
    for(int i = 0; i < m->rows * m->cols; i++)
    {
        m->data[i] = (float)(rand() % 100) / 10.0f;
    }
}
```

- `srand(time(nullptr))` – устанавливаем начальное значение (seed) для генератора случайных чисел; в нашем случае это `time(nullptr)`, что позволяет каждый раз генерировать разные последовательности;
- `rand()` – возвращает псевдослучайное число от 0 до `RAND_MAX` (обычно 32767);

4. Измерение времени выполнения алгоритмов

```
double get_time_ms()
{
    auto now = std::chrono::high_resolution_clock::now();
    auto duration = now.time_since_epoch();
    return std::chrono::duration<double, std::milli>(duration).count();
}
```

- `std::chrono::high_resolution_clock` – класс, представляющий самые точные часы в ОС (в GNU/Linux это 1 наносекунда);
- `now()` – статический метод, возвращающий текущий момент времени;
- `time_since_epoch()` – метод, возвращающий время, прошедшее с эпохи, т.е. фиксированной точки отсчёта (обычно 1970-01-01 00:00:00 UTC);
- `std::chrono::duration<double, std::milli>(duration)` – преобразует исходную длительность в миллисекунды;
- `count()` – метод, возвращающий числовое значение длительности;

5. Наивная реализация (без оптимизаций)

```
void matrix_multiply_naive(const Matrix* A, const Matrix* B, Matrix* C)
{
    int n = A->rows;
    int m = A->cols;
    int p = B->cols;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < p; j++)
        {
            float sum = 0;
            for (int k = 0; k < m; k++)
            {
                sum += A->data[i * m + k] * B->data[k * p + j];
            }
            C->data[i * p + j] = sum;
        }
    }
}
```

- **Проблема чтения данных:** Процессор никогда не читает из оперативной памяти одно число (4 байта). Он всегда берёт целый блок – кэш-линию (обычно 64 байта). Это 16 соседних чисел типа `float`.
- **Что происходит под капотом:** Когда мы пишем `A->data[i * m + k]`, происходит следующая цепочка:
 - Загрузка в кэш: Процессор проверяет, есть ли данные в кэше. Если нет (кэш-промах), он тянет из оперативной памяти (RAM) кэш-линию (64 байта), в которой находится `A[i][k]`, и кладёт её в кэш-память.

➤ Загрузка в регистр: Из этой кэш-линии процессор берёт конкретные 4 байта (число типа `float`) и копирует их в регистр.

- **Матрица A**: Мы идём по строке `A->data[i * m + k]`. Индекс `k` растёт на 1. Мы читаем данные подряд. Процессор доволен: одна загрузка из памяти — 16 полезных операций. Это называется последовательный доступ.
- **Матрица B**: Мы обращаемся к `B->data[k * p + j]`. С каждой итерацией `k` мы прыгаем на целую строку вниз ($+ p$). Если строка матрицы содержит 1024 элемента, то между `B[0][j]` и `B[1][j]` лежит 4 килобайта данных. Но кэш-линия содержит соседние элементы в строке, а нам нужен элемент в следующей строке.
- **Итог**: Мы используем только 1 число из 16 пришедших, а остальные 15 просто занимают место в кэше и выбрасываются. Когда цикл доходит до следующего `k`, нужного числа в кэше нет. Процессор вынужден стоять и ждать, пока данные придут из медленной оперативной памяти (RAM). Это называется кэш-промах. В нашем коде для матрицы `B` кэш-промах случается почти на каждой итерации внутреннего цикла.

6. Оптимизированная версия (смена порядка циклов)

```
void matrix_multiply_optimized(const Matrix* A, const Matrix* B, Matrix* C)
{
    int n = A->rows;
    int m = A->cols;
    int p = B->cols;

    memset(C->data, 0, n * p * sizeof(float));

    for (int i = 0; i < n; i++)
    {
        for (int k = 0; k < m; k++)
        {
            float aik = A->data[i * m + k];
            for (int j = 0; j < p; j++)
            {
                C->data[i * p + j] += aik * B->data[k * p + j];
            }
        }
    }
}
```

- **Матрица A**: Значение `aik` фиксировано для всего внутреннего цикла. Процессор один раз загружает его в регистр и использует `p` раз. Промахов нет!
- **Матрица B**: Мы обращаемся к `B->data[k * p + j]`. С ростом `j` мы идем строго по строке матрицы `B`.
- **Матрица C**: Мы обращаемся к `C->data[i * p + j]`. С ростом `j` мы идем строго по строке матрицы `C`.

- **Итог:** В этой версии мы прочитали кэш-линию для матрицы B, использовали первое число, а следующие 15 уже лежат в быстром кэше. Когда цикл перейдёт к $j+1$, данные уже на месте. Процессор работает на полной скорости без простоев.

7. Векторизованная версия

```
void matrix_multiply_vectorized(const Matrix* A, const Matrix* B, Matrix* C)
{
    int n = A->rows;
    int m = A->cols;
    int p = B->cols;

    memset(C->data, 0, n * p * sizeof(float));

    for (int i = 0; i < n; i++)
    {
        for (int k = 0; k < m; k++)
        {
            float aik = A->data[i * m + k];
            const float* b_row = &B->data[k * p];
            float* c_row = &C->data[i * p];

            #pragma omp simd
            for (int j = 0; j < p; j++)
            {
                c_row[j] += aik * b_row[j];
            }
        }
    }
}
```

- Векторизация – это процесс преобразования программы так, чтобы она выполняла одну и ту же операцию над целым набором данных за один такт процессора, вместо того чтобы обрабатывать каждое число по отдельности.
- `#pragma omp simd` – OpenMP директива для явного указания компилятору векторизовать цикл.
- **Без директивы:** компилятор сам решает, векторизовать цикл или нет.
- **Что проверяет компилятор:**
 - Нет зависимостей между итерациями;
 - Последовательный доступ к памяти;
 - Нет вызовов функций внутри цикла;
 - Нет сложных условных конструкций;
- **Почему наш цикл идеален для векторизации:**
 - $j+1$ итерация не зависит от j ;
 - Доступ к памяти последовательный: `c_row[0]`, `c_row[1]`, ...;
 - В цикле нет условий и вызовов функций (одна арифметическая операция);

8. Векторизованная версия с выравниванием

```
void matrix_multiply_aligned(const Matrix* A, const Matrix* B, Matrix* C)
{
    int n = A->rows;
    int m = A->cols;
    int p = B->cols;

    memset(C->data, 0, n * p * sizeof(float));

    for (int i = 0; i < n; i++)
    {
        for (int k = 0; k < m; k++)
        {
            float aik = A->data[i * m + k];
            const float* b_row = &B->data[k * p];
            float* c_row = &C->data[i * p];

            #pragma omp simd aligned(b_row, c_row: 64)
            for (int j = 0; j < p; j++)
            {
                c_row[j] += aik * b_row[j];
            }
        }
    }
}
```

- `#pragma omp simd aligned(b_row, c_row: 64)`
 - `aligned` – ключевое слово, указывающее на выравнивание;
 - `b_row, c_row` – указатели, которые выровнены;
 - `64` – размер выравнивания в байтах;
- **Что эта директива говорит компилятору:**
 - «Я гарантирую, что `b_row` указывает на память, выровненную на 64 байта»;
 - «Я гарантирую, что `b_row` указывает на память, выровненную на 64 байта»;
 - «Используй быстрые выровненные SIMD инструкции»;
- **Почему 64 байта:**
 - Максимальное выравнивание для современных процессоров;
 - Совпадает с размером кэш-линии (обычно 64 байта);
 - Работает со всеми SIMD расширениями;
- **Почему невыровненный доступ медленнее:**
 - Нужно загрузить 2 кэш-линии вместо 1 (данные пересекают границу двух кэш-линий);
 - Дополнительные такты ожидания памяти;

9. Многопоточная версия

```
void matrix_multiply_parallel(const Matrix* A, const Matrix* B, Matrix* C)
{
    int n = A->rows;
    int m = A->cols;
    int p = B->cols;

    memset(C->data, 0, n * p * sizeof(float));

    #pragma omp parallel for
    for (int i = 0; i < n; i++)
    {
        for (int k = 0; k < m; k++)
        {
            float aik = A->data[i * m + k];
            #pragma omp simd
            for (int j = 0; j < p; j++)
            {
                C->data[i * p + j] += aik * B->data[k * p + j];
            }
        }
    }
}
```

- OpenMP (Open Multi-Processing) – это стандарт и набор инструментов для параллельного программирования на языках C, C++ и Fortran.
- `#pragma omp parallel for` – директива OpenMP для распараллеливания цикла.
- **Распределение итераций:** в нашем случае статическое (итерации делятся поровну), т.к. каждая итерация требует одинакового объёма работы.
- **Почему параллелизация по i:**
 - Каждая строка `C[i]` независима от других строк;
 - Нет гонок данных (data races) – разные потоки пишут в разные строки;

Тестовая программа `tests/test_matrix.cpp`

```
#include "../include/matrix_ops.h"
#include <iostream>
#include <iomanip>

int main()
{
    const int N = 1024;
    std::cout << "===== Тестирование умножения матриц " << N << "x" << N << " "
    << "=====\n\n";

    // Создание матриц
    Matrix* A = create_matrix(N, N);
    Matrix* B = create_matrix(N, N);
    Matrix* C = create_matrix(N, N);

    init_random(A);
    init_random(B);
}
```

```

double start = 0.0;
double end = 0.0;

// 1. Наивная (без оптимизаций)
start = get_time_ms();
matrix_multiply_naive(A, B, C);
end = get_time_ms();
double time_naive = end - start;
std::cout << "1. Наивная (без оптимизаций):\t" << time_naive << " ms\n";

// 2. Оптимизированная (смена порядка циклов)
start = get_time_ms();
matrix_multiply_optimized(A, B, C);
end = get_time_ms();
double time_opt = end - start;
std::cout << "2. Оптимизированная (смена порядка циклов):\t" << time_opt << «
ms\n»;

// 3. Векторизованная
start = get_time_ms();
matrix_multiply_vectorized(A, B, C);
end = get_time_ms();
double time_vec = end - start;
std::cout << "3. Векторизованная:\t" << time_vec << " ms\n";

// 4. Векторизованная с выравниванием
start = get_time_ms();
matrix_multiply_aligned(A, B, C);
end = get_time_ms();
double time_align = end - start;
std::cout << «4. Векторизованная с выравниванием:\t» << time_align << « ms\n»;

// 5. Многопоточная
start = get_time_ms();
matrix_multiply_parallel(A, B, C);
end = get_time_ms();
double time_par = end - start;
std::cout << "5. Многопоточная:\t" << time_par << " ms\n";

// Очистка
free_matrix(A);
free_matrix(B);
free_matrix(C);

return 0;
}

```

Файл для сборки Makefile

```

CXX = g++
CXXFLAGS = -Wall -O2 -march=native -fopenmp -fPIC
VECFLAGS = -fopt-info-vec-optimized -fopt-info-vec-missed
INCLUDES = -I./include

all: lib/libmatrix.so tests/test_matrix

lib/libmatrix.so: src/matrix_ops.cpp | lib
    $(CXX) $(CXXFLAGS) -shared $(INCLUDES) $< -o $@

lib/libmatrix_vecinfo.so: src/matrix_ops.cpp | lib

```

```
$(CXX) $(CXXFLAGS) $(VECFLAGS) -shared $(INCLUDES) $< -o $@

tests/test_matrix: tests/test_matrix.cpp src/matrix_ops.cpp | tests
    $(CXX) $(CXXFLAGS) $(INCLUDES) $^ -o $@

clean:
    rm -rf lib/*.so tests/test_matrix

run: all
    LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH ./tests/test_matrix

.PHONY: all clean run
```

Переменные:

- CXX – стандартное имя переменной для C++ компилятора;
- CXXFLAGS – стандартное имя для флагов компиляции C++;
- INCLUDES – имя пути к заголовочным файлам;
- VECFLAGS – имя для флагов диагностики;

Флаги компиляции:

- -Wall (Warnings all) – включить все основные предупреждения;
 - -O2 – оптимизация 2-ого уровня (стандартная оптимизация);
 - Разворачивание небольших циклов (Loop unrolling)
 - Встраивание маленьких функций (Inlining)
 - Удаление мёртвого кода (Dead Code Elimination)
 - -march=native (machine architecture native) – оптимизация для текущего процессора;
 - -fopenmp – включение поддержки многопоточности (позволяет использовать #pragma omp ..., линкует с библиотекой OpenMP, без этого флага директивы OpenMP игнорируются);
 - -fPIC (Position Independent Code) – позиционно-независимый код (для создания динамических библиотек; без -fPIC - ошибка при линковке .so);
 - -shared – создание разделяемой библиотеки (.so), а не исполняемого файла;
 - -fort-info-vec-optimized – флаг, выводящий информацию о том, какие циклы компилятор смог векторизовать;
 - -fort-info-vec-missed – флаг, выводящий информацию о том, почему он не смог векторизовать определённые циклы;
- I – добавить директорию для поиска заголовочных файлов (чтобы компилятор знал, где искать описания функций);

Специальные переменные автоматических правил:

- \$< - первая зависимость;

- `$@` - имя цели;
- `$$` - все зависимости;

Цели:

- `all` – основная цель по умолчанию.
 - Вызывается, если просто написать `make` (без аргументов);
 - Зависит от двух целей: `lib/libmatrix.so` и `tests/test_matrix`;
- `lib/libmatrix.so` – динамическая библиотека.
 - `| lib` – папка `lib` должна существовать (без этого `make` пересобирает бы библиотеку каждый раз, когда в папке `lib` что-то меняется);
- `lib/libmatrix_vecinfo.so` – версия с диагностикой.
 - Добавляет флаги `-fopt-info-vec-optimized` и `-fopt-info-vec-missed`;
- `tests/test_matrix` – тестовая программа.
- `clean` – очистка.
 - Удаляет рекурсивно и без подтверждения (`-rf`) все `.so` файлы в папке `lib` и исполняемый файл `tests/test_matrix`;
- `run` – сборка и запуск тестовой программы.
 - `LD_LIBRARY_PATH` – переменная окружения для поиска `.so` библиотек;
 - `$$` - в Makefile `$` - это специальный символ; `$$` превращается в `$` при передаче в shell;
 - Добавляем `./lib` в путь поиска библиотеки;
- `.PHONY` – фальшивые цели.
 - Указывает, что эти цели – не файлы, а названия команд (например, если бы существовал файл с именем `clean`, `make clean` не работал бы);

Сборка и запуск

<code>make</code>	<code>// сборка всего проекта</code>
<code>make lib/libmatrix_vecinfo.so</code>	<code>// сборка библиотеки для отладки</code>
<code>make run</code>	<code>// сборка и запуск тестовой программы</code>
<code>make clean</code>	<code>// очистка</code>

Вывод программы и анализ полученных результатов

===== Тестирование умножения матриц 1024x1024 =====	
1. Наивная (без оптимизаций):	8487.103 ms
2. Оптимизированная (смена порядка циклов):	575.844 ms
3. Векторизованная:	273.818 ms
4. Векторизованная с выравниванием:	240.950 ms
5. Многопоточная:	69.372 ms